Automatic Prefix Caching

Introduction

Automatic Prefix Caching (APC in short) caches the KV cache of existing queries, so that a new query can directly reuse the KV cache if it shares the same prefix with one of the existing queries, allowing the new query to skip the computation of the shared part.

Note

Technical details on how vLLM implements APC can be found here.

Enabling APC in vLLM

Set `enable_prefix_caching=True` in vLLM engine to enable APC. Here is an example:

examples/offline_inference/automatic_prefix_caching.py

Example workloads

We describe two example workloads, where APC can provide huge performance benefit:

- Long document query, where the user repeatedly queries the same long document (e.g. software manual or annual report) with different queries. In this case, instead of processing the long document again and again, APC allows vLLM to process this long document *only once*, and all future requests can avoid recomputing this long document by reusing its KV cache. This allows vLLM to serve future requests with much higher throughput and much lower latency.
- Multi-round conversation, where the user may chat with the application multiple times in the same chatting session. In this case, instead of processing the whole chat history again and again, APC allows vLLM to reuse the processing results of the chat history in all past rounds of conversation, allowing vLLM to serve future requests with much higher throughput and much lower latency.

Limits

APC in general does not reduce the performance of vLLM. With that being said, APC only reduces the time of processing the queries (the prefilling phase) and does not reduce the time of generating new tokens (the decoding phase). So APC does not bring performance gain when vLLM spends most of the time generating answers to the queries (e.g. when the length of the answer is long), or new queries do not share the same prefix with any of existing queries (so that the computation cannot be reused).

October 17, 2025